GITCHECK

Pre-Commit Verification Gate & Comprehension Harness for AI-Assisted Code

PITCH DECK & PRODUCT SPECIFICATION

> IMPORTANT
>
> **THE MACRO PARADIGM SHIFT**
>
> ## AI Solved the Drafting Bottleneck. It Created a Review Crisis.
>
> AI coding assistants generate **30% more code**, but shipped throughput remains flat because **PR review times have surged by 49%**. **GitCheck** introduces an author-side pre-commit comprehension gate that verifies developer understanding via counterfactual traps, intercepts AI copy-pasting, eliminates rework loops, and attaches an auditable defense card before pull requests hit human review.

SLIDE 1: THE MACRO PROBLEM (EMPIRICAL DEVELOPER DATA)

`Validation`

+30%	**+49%**	**0%**
Code Volume Generated	**PR Review Latency Spike**	**Net Gain in Shipped Features**

The "Two Clocks" Phenomenon

AI coding tools (Cursor, Copilot, Claude Code) optimize **Clock 1 (Drafting Speed)** by dumping unvetted diffs into repositories. But software delivery velocity is constrained by **Clock 2 (Defensible Decisions)**, where human reviewers must verify architecture, security, and edge cases.

Comprehension Debt

Developers submit code they cannot explain. When Principal Engineers question abstractions or duplicate logic, authors fall back on unmediated prompt-relaying, creating multi-day review ping-pong and management escalations.

SLIDE 2: WHO FEELS THE FRICTION? `User Hierarchy`

Role / Stakeholder	Primary Pain Point	Root Cause
Software Engineers *(Authors)*	PR embarrassment, multi-day review loops, and rework.	Lack of mental model for agent-generated diffs; unable to defend choices during review.
Principal Engineers *(Reviewers)*	Burnout from acting as manual linters and architecture watchdogs.	Flooded with bloated diffs that duplicate existing repo utilities and drift from system patterns.
VP of Engineering *(Buyer)*	Zero ROI on AI tool budget; release velocity halted by PR bottlenecks.	Local drafting speed creates operational gridlock at the review gate (*Theory of Constraints*).

SLIDE 3: THE SOLUTION — `GITCHECK` `Comprehension Gate`

GitCheck is a local CLI harness and cloud anti-rework system that runs at `git commit`. It verifies developer comprehension via Socratic counterfactual traps, intercepts AI copy-pasting, and attaches an auditable defense card to the PR.

1. Hybrid Socratic Verification (2+1 Model)

Generates 2 descriptive questions testing architectural trade-offs and edge cases, plus 1 counterfactual trap (Yes/No) with an intentional false premise. Pushing `Y` to a plausible falsehood instantly catches blind agreement.

2. AI Copy-Paste Detection & Micro-Implementation Probes

Analyzes developer explanations for synthetic AI boilerplate. If ChatGPT/Claude text is detected, GitCheck challenges the author with a tricky micro-implementation question targeting exact parameters or exceptions in the diff.

3. 80% Acceptance Threshold & Author Accountability Gate

Requires $\geq 80\%$≥ 80% combined score and zero traps triggered to pass. If a trap is triggered or score falls below 80%, GitCheck surfaces the false premise, presents a 50–100 word architectural explanation, and prompts the author to explicitly accept responsibility (`[y/N]`).

4. Anti-Rework PR Defense Card & History Sync

Writes an auditable markdown defense card (`GITCHECK_DEFENSE_CARD.md`) and Git trailer. Automatically syncs context to Supabase and a React single-page dashboard, giving reviewers historical context on prior PR rejections to prevent rework loops. *(Quiz grades remain confidential; only architectural defenses are published)*.

SLIDE 4: DEVELOPER COMPREHENSION SCORE (DCSDCS) `Mathematical Engine`

We evaluate comprehension mathematically to prevent developers from faking understanding via external web chats:

$$\text{DCS} = (w_{\text{arch}} \cdot S_{\text{arch}} + w_{\text{edge}} \cdot S_{\text{edge}} + w_{\text{trap}} \cdot T) - \lambda_{\text{cheat}}$$

DCS = (warch ⋅ Sarch + wedge ⋅ Sedge + wtrap ⋅ T) − λcheat

- **Architectural Rationale (S_{arch} Sarch):** Evaluates depth of explanation regarding pattern choices and abstractions ($w_{\text{arch}} = 0.35$warch = 0.35).
- **Edge-Case & Failure Resilience (S_{edge} Sedge):** Evaluates coverage of error handling, null boundaries, timeouts, and concurrency ($w_{\text{edge}} = 0.35$wedge = 0.35).
- **Trap Resilience (TT):** Penalizes false premise agreement ($w_{\text{trap}} = 0.30$wtrap = 0.30):
  $$T = I(y_{\text{user}} = y_{\text{expected}}), \quad \text{where } y_{\text{expected}} = \text{False}$$
  T = I(yuser = yexpected), where yexpected = False Pushing `Yes` triggers $T = 0$T = 0 and activates the mandatory review gate.
- **Anti-Cheating Behavioral Penalty (λ_{cheat} λcheat):**
  $$\lambda_{\text{cheat}} = \begin{cases} 0 & \text{Human explanation or passed tricky counter-probe} \\ 0.70 & \text{AI copy-paste detected AND failed micro-implementation counter-probe} \end{cases}$$
  λcheat = {00.70Human explanation or passed tricky counter-probeAI copy-paste detected AND failed micro-implementation counter-probe
- **Passing Gate Requirement:** $\text{DCS} \geq 0.80 \quad \wedge \quad T = 1$DCS ≥ 0.80 ∧ T = 1

SLIDE 5: THE 60-SECOND TERMINAL EXPERIENCE `Developer Workflow`

```
bash
$ git-check run

┌──────────────────────────────────────────────────────────────┐
│ GitCheck • Pre-Commit Verification Gate                      │
│ Comprehension Harness for AI-Assisted Code                   │
└──────────────────────────────────────────────────────────────┘

Branch: feat/PROJ-402-payment-webhook-retry | Ticket: PROJ-402
Author: Milind Mane <milind.mane@company.internal>
Files changed: 3 | +84 -12

┌─ Historical PR Context (Anti-Rework) ────────────────────────┐
│ Previous context found for ticket PROJ-402 (1 past iteration).│
│ Most recent rejection: In-memory retry failed during cluster restarts.│
└──────────────────────────────────────────────────────────────┘

Analyzing diff & AST via GEMINI-1.5-FLASH Verification Engine...

Question 1 / 3: [Architectural Rationale]
Why was Redis chosen for storing webhook retry state instead of an in-memory queue?
Your explanation: Redis provides distributed state persistence across service
restarts and multiple cluster nodes.

Question 2 / 3: [Edge Cases & Failure Modes]
What occurs when Redis connection pool times out during webhook receipt?
Your explanation: According to best practices in distributed systems, when
connection pool exhaustion happens...
⚠️  AI-Generated Answer Detected! [Textbook boilerplate detected]
┌─ AI Copy-Paste Interceptor ──────────────────────────────────┐
│ Tricky Micro-Implementation Counter-Question:                │
│ "Looking directly at payment_webhook.py line 48: what exact exception is  │
│  caught on Redis timeout, and what fallback HTTP status code is returned?" │
└──────────────────────────────────────────────────────────────┘
Your micro-implementation answer: ConnectionError; it logs an alert and returns
HTTP 503 Service Unavailable.
✔ Tricky question passed! Author verified direct code knowledge.

Question 3 / 3: [Counterfactual Trap Check - Yes/No]
"Does line 62 silently drop failed webhooks if the signature header is empty?"
Does this statement accurately describe what your changes do? (y/n): n
✔ Counterfactual Trap Avoided! (Expected: No; Code raises InvalidSignatureError).

✔ Verification Passed (88.5%)! Comprehension Verified.

┌─ Pre-Commit Gate Completed Successfully ─────────────────────┐
│ • Summary Note: Added to PR defense artifact                 │
```

```
│ • Defense Card: .gitcheck/GITCHECK_DEFENSE_CARD.md                          │
│ • Database: Synced to Supabase (https://xyz.supabase.co)                    │
│ • Review Dashboard: https://gitcheck-review.vercel.app/ticket/PROJ-402      │
└─────────────────────────────────────────────────────────────────────────────┘
```

SLIDE 6: HOW WE BUILT IT `Tech Stack`

Component	Technology Stack	Implementation Details
CLI Core	Python 3.10+, Click, Rich	Lightweight terminal UI executing in sub-second local runtime. Installed via `pip`.
AST & Diff Parser	Native `ast`, Multi-Language Regex	Parses staged diffs, extracts mutations (functions, classes, exceptions, imports, returns).
Adversarial Engine	Gemini 1.5 Flash / Claude Haiku / GPT-4o-mini	Generates 2 design probes and 1 counterfactual trap per staged diff context.
Anti-Cheating Engine	LLM Classifier + Micro AST Probes	Detects generic AI boilerplate and validates micro-implementation details.
Anti-Rework Cloud Store	Supabase PostgreSQL + RLS	Centralizes commit summaries and rejection context with fast indexing by ticket and commit hash.
Web Review Dashboard	React 18, Vite, TypeScript, Tailwind CSS	Single-page commit review interface with timeline of previous PR rejections, deployed on Vercel.
Git Hook Integration	Native `.git/hooks` (`pre-commit`)	Non-blocking pre-commit hook with emergency bypass logging (`--no-verify` / `--skip`).

SLIDE 7: COMPETITIVE LANDSCAPE `Market Differentiation`

Solution Category	Key Examples	Primary Limitation	GitCheck Advantage
PR Review Bots	CodeRabbit, PR-Agent, Copilot PR	Downstream on GitHub; adds comment noise; author copy-pastes bot feedback into Cursor.	**Pre-Commit Gate:** Shifts verification left into developer's terminal before PR creation.
General LLMs	ChatGPT, Claude Chat, Cursor inline	Passive explanations; acquiescence bias validates user's false premises.	**Active Recall:** Adversarial traps and counter-probes force human inspection.
Static Linters	SonarQube, Semgrep, ESLint	Checks syntax and CVEs; completely blind to intent, trade-offs, and architecture.	**Semantic & AST Fit:** Verifies intent, architectural choices, and edge-case handling.
PR Cycle Dashboards	LinearB, Jellyfish, DX	Measures cycle time post-mortem after bottleneck has already occurred.	**Anti-Rework Context:** Surfaces prior PR rejection history to avoid repetitive rework loops.

SLIDE 8: ROI & GO-TO-MARKET STRATEGY `Business Case`

Bottom-Up Developer Adoption

- Distributed as a free open-source CLI (`pip install git-check`).
- Individual developers use it to prevent PR embarrassment, clarify commit intent, and speed up merge approvals.
- Strict author confidentiality: comprehension grades are private; only verified architectural summaries are shared.

Enterprise Governance & Anti-Rework Tier

- Monetized via centralized anti-rework management: tracking comprehension health, surfacing historical rejection timelines, and eliminating repetitive rework across 100+ dev teams.
- Integrates directly into corporate Jira/Linear workflows via automatic ticket detection.

> TIP

> **The Bottom Line:** Cutting PR review latency from **48h to 4h** saves **12+ hours of Principal Engineer time per week**. **ROI: ~15x per developer seat annually**.